

ESLtopia — Master Test Plan

Worksheet Generator & Classroom Management Platform · Test Plan v1.6 · 2026-07-06

Application	ESLtopia (English worksheet generator + classroom management)
Production URL	https://esltopia.vercel.app
Repository	github.com/JovovicMiljan89/ESLtopia
Document version	1.6
Status	Active
Related documents	Test Cases Specification, Test Scenario Coverage Checklist, docs/qa/test-coverage-todo.md

v1.6 revision: Brought cross-browser coverage (§4.2 previously listed this as out of scope) in for Firefox and WebKit via two new `playwright.config.ts` projects. Rather than run all 144 cases under all three engines, the new projects exclude `classes.spec.ts` and `profile.spec.ts` (pure REST/RLS assertions with no page rendering — identical results regardless of engine) and `tests/visual/**` (baselines only exist for Chromium). All 234 resulting Firefox/WebKit executions pass; the one failure and one flake seen in an initial full run were confirmed by isolated reruns to be the suite's existing parallel-load flakiness (§10/§11), not a real cross-browser defect — no app or test code needed a fix. CI (`daily-tests.yml`) now installs and runs all three browsers; `npm test` does too, with `npm run test:chromium` added for a fast single-browser loop during local development.

v1.5 revision: Closed the three high-priority items in `test-coverage-todo.md` : (1) Classes tab UI click-through — new `tests/app/classes-ui.spec.ts` (CLSUI-01–06) drives the "add class" form, roster add/remove inputs, and the delete-class button, which previously only had REST coverage; (2) XSS/HTML-injection resilience — CLSUI-05/06 inject markup into class/student names, and RECUI-09 into the notes field, confirming React's default escaping holds; (3) rate-limiting behavior on `/auth/v1/token` and `/auth/v1/recover` — LOGIN-17 and FPW-08 exercise a small burst of repeated requests. That last one is a documentation win more than a security guarantee: as observed 2026-07-06, production shows **no lockout or 429 within a 4–5 request burst on either endpoint** — the tests lock in the invariant that failed attempts never succeed and the server never 500s, but do not confirm brute-force protection actually exists. See §11 for the resulting risk note. Total: 135 → 144 cases across 14 → 15 spec files.

v1.4 revision: Brought `tests/visual/visual-regression.spec.ts` into scope — 4 new pixel-diff cases (VIS-01–04) covering the login screen, dashboard, worksheet generator settings, and PDF preview modal, wired into CI via `daily-tests.yml` and refreshed via a dedicated `update-visual-baselines.yml` workflow. This suite existed in the repo but was undocumented in this plan (§4.2 previously listed visual regression as out of scope). Also documented two test cases that were already in the codebase but missing from the Test Case Specification: **REG-11** (duplicate-email signup anti-enumeration) and **CLS-08** (RLS blocks inserting a class row with a forged `owner_id`) — every case after each was renumbered by one to keep IDs sequential. Total: 129 → 135 cases across 13 → 14 spec files.

v1.3 revision: Added RECUI-08 (128 → 129 total), closing the record-sync-failure gap the v1.1 review surfaced. Both scenario gaps that review found are now automated. A standing `test-coverage-todo.md` now tracks everything else still open, consolidated from the Test Scenario Coverage Checklist.

v1.2 revision: Added 2 test cases (SCHT-14, SCHT-15 — 126 → 128 total) closing the teacher-removal cascade gap the v1.1 review surfaced: a populated teacher's classes/records are now verified to actually cascade-delete on removal, and the confirm dialog's warning text is verified against the fix applied in v1.1.

v1.1 revision: `EnglishGenerator.jsx` was split from a single ~4,750-line file into 8 focused modules (\$2 updated below); a code review pass fixed one critical data-loss-warning gap, two silent-error-handling bugs, a stale-closure bug, a biased randomization function, and deduplicated a test helper (see the Test Case Specification's revision note for details). No test cases were added or removed at that point — the 126-case suite was unchanged in count until v1.2 above.

1. Introduction & Purpose

This document defines the testing strategy for ESLtopia, a React/Vite single-page application backed by Supabase (Postgres, Auth, Edge Functions) and deployed to Vercel. It covers what is tested, how, where, with what data, and what "done" means for a release. It is the reference companion to the **Test Cases Specification** (detailed, step-by-step cases mapped to the automated suite) and the **Test Scenario Coverage Checklist** (the full list of scenarios that should be covered, including gaps not yet automated).

2. Application Under Test — Overview

ESLtopia lets teachers (and schools managing teachers) generate printable English-language worksheets for grades 1–6 across 24 topics and five exercise types, preview/print/download them as a formatted A4 PDF, and manage classes, students, and per-student attendance/grade/payment records. Three roles exist: teacher, school, and superadmin, enforced end-to-end via Postgres Row-Level Security (RLS).

Layer	Technology
Frontend	React 18, Vite 6, split into focused modules under <code>src/</code> : <code>EnglishGenerator.jsx</code> (main orchestrator, ~950 lines), <code>AuthScreen.jsx</code> , <code>AdminPanel.jsx</code> , <code>EmployeesPanel.jsx</code> , <code>LandingPage.jsx</code> , <code>PdfPreviewModal.jsx</code> , <code>WorksheetTasks.jsx</code> , plus data/style modules (<code>worksheetContent.js</code> , <code>profileHelpers.js</code> , <code>styles.js</code>)
Backend	Supabase (Postgres + RLS, GoTrue Auth, Edge Functions on Deno)
Email	Gmail SMTP (confirmation / invite / password-reset mail)
Hosting	Vercel (manual <code>vercel --prod</code> deploys; not auto-triggered by git push)
Test framework	Playwright + TypeScript. UI/API/regression tests run on Chromium, Firefox, and WebKit; visual-regression screenshots remain Chromium-only, rendered on CI's <code>ubuntu-latest</code> runner — regenerating them on a local dev machine produces baselines that will fail in CI due to font-rendering differences.

3. Objectives

- Verify that every user-facing flow (auth, worksheet generation, PDF preview, classes, records, admin/school management) behaves correctly for each role.
- Verify that Postgres RLS policies and DB triggers correctly enforce data isolation and prevent privilege escalation.
- Catch regressions — both functional and visual — before they reach real users of a live, low-traffic production app with no staging environment.
- Keep the suite fast and deterministic enough to run before every deploy.

4. Scope

4.1 In scope

- Authentication: signup, login, logout, forgot/reset password, session persistence, account status gating (pending/inactive).

- Authorization: teacher / school / superadmin role boundaries, RLS isolation, privilege-escalation prevention, JWT claim correctness.
- Superadmin Admin Panel: user visibility, role changes, profile deletion.
- School → Teachers management (Employees panel + edge functions), currently gated behind `FEATURE_SCHOOL_TEACHERS`.
- Classes: CRUD, student roster management, RLS isolation (including INSERT-time ownership forgery), cascade delete of records.
- Records: attendance / grades / payment tracking UI, per-student profile modal, notes, debounced Supabase sync.
- Worksheet Generator: topic/grade selection, task-type switching, worksheet generation logic.
- PDF Preview Modal: open/close (button + Escape), answer-key toggle, regenerate ("New set"), print/download triggers.
- **Visual regression:** pixel-diff screenshots of four key screens (login, dashboard, worksheet generator settings, PDF preview modal) via Playwright's `toHaveScreenshot()`, tolerant to a 2% diff-pixel ratio to absorb anti-aliasing noise.
- **Cross-browser:** Chromium, Firefox, and WebKit for every UI/API test except the pure-REST files (`classes.spec.ts` , `profile.spec.ts`) and visual regression — see §13.

4.2 Out of scope (for this suite)

- Mobile browsers / mobile viewport emulation — desktop Chromium/Firefox/WebKit only.
- Visual regression of the *printed* worksheet PDF output itself (print CSS / `window.print()` behavior) — the four in-scope visual snapshots cover app screens, not the print-rendered worksheet.
- Load, stress, and performance testing (no defined SLA; ~10-user portfolio app).
- Accessibility audit beyond incidental keyboard support (Escape-to-close).
- Actual email content/deliverability verification (tests use the admin API instead of reading an inbox).
- Native mobile apps (none exist).
- Unit/component-level testing — the project deliberately uses a single E2E layer (see §6).

See the Test Scenario Coverage Checklist for the authoritative, itemized list of what is and isn't automated within the in-scope areas above.

5. Test Levels & Types

Type	Description	Example
UI / E2E	Drives the real browser against production, asserting on rendered DOM state.	Toggling the PDF answer-key switch; marking a student present in the Records tab.
API / REST	Calls Supabase Auth/REST endpoints directly with <code>request</code> , bypassing the UI, to assert on RLS and trigger behavior precisely.	PATCH a profile's <code>role</code> column and confirm the DB trigger rejects it.
Security / RLS	Confirms Postgres policies isolate data between roles/owners and block privilege escalation, on both read and write paths.	Teacher B cannot read Teacher A's class; Teacher B cannot insert a class row forging Teacher A's <code>owner_id</code> .
Visual / Pixel-diff	Screenshot comparison against a checked-in baseline (<code>toHaveScreenshot</code>) to catch unintended visual drift that no DOM assertion would think to check for.	A worksheet generator button losing its background color; the login card's layout shifting.
Regression	The full suite, run as a whole before any production deploy.	144 cases across 15 spec files; ~378 total executions once the Firefox/WebKit projects are counted.

6. Test Strategy

The suite is intentionally a single E2E layer rather than split unit/integration/E2E layers: the app is a single component file with no isolated business-logic modules to unit-test in isolation, and the primary risk surface is Supabase RLS/trigger behavior plus real user flows — both are best verified against the real stack. Where a UI click would just be a thin wrapper over a REST call already covered elsewhere (e.g. class/record CRUD), tests call the REST API directly for precision and speed; where the click handler itself is the thing under test and has no other coverage (e.g. Records tab toggles, PDF modal interactions), tests drive the real UI.

Visual regression is the one deliberate exception to that DOM-assertion strategy: four key screens are compared pixel-for-pixel against a checked-in baseline instead, to catch incidental layout/style regressions that no targeted assertion would think to check for. Baselines are only valid when generated on the same OS/font stack as CI (`ubuntu-latest`) — regenerating them on a local dev machine (macOS/Windows) produces baselines that will fail in CI. To (re)generate baselines after an intentional UI change, run the "Update Visual Regression Baselines" GitHub Actions workflow (`workflow_dispatch`) and commit the resulting artifact over `tests/visual/visual-regression.spec.ts-snapshots/`.

Tests run against **production** (`https://esltopia.vercel.app`), not a staging environment — there isn't one. This is a deliberate tradeoff for a low-traffic portfolio app; see §11 (Risks) for how this is made safe.

7. Test Environment

Item	Detail
Target	Production — <code>https://esltopia.vercel.app</code> (override with <code>BASE_URL</code> env var)
Backend project	Supabase <code>jogbbmssrdlujzlxwiji</code>
Browser	Chromium, Firefox, and WebKit (all headless), via Playwright's <code>chromium / firefox / webkit</code> projects
Parallelism	<code>fullyParallel: true</code> locally; <code>workers: 1</code> on CI
Secrets	<code>.env.test.local</code> (gitignored): <code>VITE_SUPABASE_URL</code> , <code>VITE_SUPABASE_ANON_KEY</code> , <code>SUPABASE_SERVICE_ROLE_KEY</code>
Feature flags	<code>FEATURE_SCHOOL_TEACHERS</code> — gates the school→teachers edge-function tests until that feature is deployed; currently set to <code>1</code> in CI
Visual baselines	<code>tests/visual/visual-regression.spec.ts-snapshots/*.png</code> , <code>Linux-rendered (ubuntu-latest)</code> ; regenerate only via the "Update Visual Regression Baselines" workflow, never locally

8. Test Data Management

- All test accounts use the `@example.test` domain via a shared `uniqueEmail(prefix)` helper, so they're unambiguously identifiable as fixtures.
- Fixture users are created pre-confirmed via the Supabase Admin API (`createConfirmedUser`) rather than the real signup+email-confirmation flow, for speed and determinism.
- A global teardown (`tests/global-teardown.ts`) purges every `@example.test` user after each run; cascading deletes remove their classes and records automatically.
- Tests that mutate shared state (e.g. Records tab) create their own isolated fixtures (a fresh class per test) rather than sharing one, because the suite runs fully parallel and shared mutable fixtures would race.

9. Entry Criteria

- Feature branch merged/pushed to `main` and, if the change is user-facing, deployed to production (`vercel --prod -yes` — not automatic on push).
- `.env.test.local` present with valid Supabase credentials.
- No known Supabase outage or maintenance window in progress.

10. Exit Criteria

- All tests pass, or fail only with a previously-known, documented flake (currently: two auth specs occasionally need their configured retry).
- No new browser console errors observed during manual/agent-driven verification of the changed feature.
- New features ship with either automated coverage or an explicit, reviewed decision to defer it (recorded in the Test Scenario Coverage Checklist as a gap).
- UI changes to any of the four visually-regression-tested screens ship with refreshed baselines (via the update-baselines workflow), not a suppressed/ignored diff.

11. Risks & Mitigations

Risk	Mitigation
Tests run against production — a bug in a test could affect real data.	All fixture data is confined to the <code>@example.test</code> domain and purged by global teardown; RLS means fixture users can never touch real users' rows.
Debounced writes (e.g. Records autosave, 800ms) have no visible "saved" indicator.	Persistence-sensitive tests explicitly wait out the debounce window, then reload and re-read from the server rather than trusting client state.
Fully parallel execution can race tests that share mutable fixtures.	Each test that mutates shared-looking state (classes, records) creates its own isolated fixture instead of reusing one across a describe block.
Frontend deploys are not automatic on git push.	Documented explicitly; a production deploy step (<code>vercel --prod --yes</code>) with bundle-hash verification is a required part of shipping a UI change before UI tests are re-run against it.
Email-dependent specs (registration, school-teachers) can flake on SMTP timing.	Those describe blocks set <code>retries: 2</code> .
A feature (school-teachers) is code-complete but not yet deployed.	Its edge-function tests are marked <code>test.skip</code> with an explicit reason, not silently omitted.
Visual baselines generated on the wrong OS produce constant false-positive diffs.	Baselines are only ever (re)generated via the CI-only "Update Visual Regression Baselines" workflow (<code>ubuntu-latest</code>), never on a contributor's local machine.
No brute-force/rate-limit protection was observed on the login and password-recovery endpoints during a small test burst (2026-07-06) — a wrong password or a repeated recovery request neither locks out nor gets throttled within 4–5 attempts.	Not itself mitigated by the test suite — this is a product-level observation, not a testing gap. Flagged here so it's a deliberate, tracked risk rather than a silent unknown; see LOGIN-17 / FPW-08 in the Test Case Specification and the corresponding PARTIAL row in the Scenario Checklist §1.
Running UI/API tests across three browser engines roughly triples signup/email/RLS-fixture traffic against the real Supabase project per full suite run, versus Chromium alone.	The two pure-REST spec files (no page rendering, so the engine can't matter) and the visual suite (Chromium-only baselines) are excluded from the Firefox/WebKit projects specifically to keep this increase bounded rather than a flat 3x on everything.

12. Roles & Responsibilities

Role	Responsibility
------	----------------

Developer / test author	Writes and maintains specs alongside feature code; runs the full suite before a production deploy.
Reviewer (future collaborators)	Reviews new specs for adherence to the conventions in §13 before merge.

13. Conventions & Tooling

- Shared helpers: `tests/helpers/cleanup.ts` (Supabase admin operations), `tests/helpers/ui.ts` (`loginToApp`), and `tests/helpers/edgeFunctions.ts` (`invokeEdgeFunction` — extracted from two byte-identical copies previously duplicated in `school-teachers.spec.ts` and `superadmin.spec.ts`).
- One `test.describe` per logical feature area; each describe block sets up its own fixtures.
- Non-auth feature specs live under `tests/app/` ; auth-related specs live under `tests/auth/` ; visual-regression specs live under `tests/visual/` .
- Cross-browser: the `firefox` and `webkit` projects in `playwright.config.ts` run everything the `chromium` project does except `classes.spec.ts` , `profile.spec.ts` (pure REST/RLS assertions — no page is ever rendered, so the engine can't affect the result), and `tests/visual/` (baselines are Chromium-only). Run just one browser locally with `npm run test:chromium` , or the other two with `npm run test:cross-browser` ; CI runs all three via plain `npm test` .
- Visual-regression tests assert one checked-in baseline each via `toHaveScreenshot()` with `maxDiffPixelRatio: 0.02` , and run in serial mode with a fixed 1280×900 viewport for determinism.
- `npx playwright test --list` is run before trusting any new spec parses correctly.

14. Deliverables

- This Test Plan.
- Test Cases Specification — every automated test, with steps and expected results.
- Test Scenario Coverage Checklist — the full scenario inventory, automated and not.
- `docs/qa/test-coverage-todo.md` — a standing, prioritized backlog of untested scenarios, kept in sync with the Scenario Checklist's gaps.
- The automated spec files themselves, under `tests/` .
- Visual regression baselines (`tests/visual/visual-regression.spec.ts-snapshots/`) and the `update-visual-baselines.yml` workflow used to refresh them.
- The Playwright HTML report, generated per run (`npx playwright show-report`).